

closure_evidence_20260925

BOB-244 closure evidence

Revision: 1 Last modified: 2026-09-25T17:30:00Z

What was reported

The pre-build sweep's CM-GITIGNORE-SWALLOW-GUARD gate (§11.4.201(6), BOB-212) refused, citing 4 untracked files under `.specify/extensions/superspec/scripts/` that looked first-party (`e2e-agent-claude.sh`, `e2e-smoke.sh`, `validate-extension-metadata.py`, `validate-release-archive.py`), silently swallowed by the directory-level `.gitignore:218` rule for `.specify/extensions/superspec/`.

Investigation

Per §11.4.124 (investigate before removing/silently-ignoring) and §11.4.6 (no guessing), read all 4 files in full. All four are clearly the upstream WangX0111/superspec project's OWN end-to-end/CI/validation tooling for testing the superspec extension itself (e.g. `e2e-smoke.sh`'s own header: "End-to-end smoke test for the superspec extension").

The project already vendors this exact upstream via a proper git submodule at `superspec/` (`.gitmodules: url = git@github.com:WangX0111/superspec.git`). Compared the 4 flagged files against their counterparts in that submodule:

```
$ diff -q superspec/scripts/e2e-agent-claude.sh .specify/
extensions/superspec/scripts/e2e-agent-claude.sh
$ diff -q superspec/scripts/e2e-smoke.sh .specify/extensions/
superspec/scripts/e2e-smoke.sh
$ diff -q superspec/scripts/validate-extension-
metadata.py .specify/extensions/superspec/scripts/validate-
extension-metadata.py
```

```
$ diff -q superspec/scripts/validate-release-archive.py .specify/
extensions/superspec/scripts/validate-release-archive.py
```

All four `diff -q` invocations produced **no output** (byte-identical, exit 0 for all four). Conclusion: these are genuine, confirmed vendored duplicates of already-properly-tracked content — not first-party, undiscovered work. The `.gitignore:218` rule excluding the whole `.specify/extensions/superspec/` directory (a nested vendored checkout of the same upstream, carrying its own `.git` `gitdir` pointer) is correct.

Root cause of the false-positive

`scripts/pre_build/check_gitignore_swallow.sh`'s `EXCLUDED_PATH_PREFIXES` array already has a precedent mechanism for exactly this class (root- relative prefixes a bare directory-name denylist can't express — the array already excluded `constitution/`, `superspec/`, and `config/download-proxy/src/`, the last one explicitly documented as “a DUPLICATE source tree”). `.specify/extensions/superspec/` was simply missing from that list — a gap, not a design flaw.

Fix

RED confirmed:

```
$ bash scripts/pre_build/check_gitignore_swallow.sh .
BOB-212 swallow-guard: REFUSED – 4 first-party source file(s) are
silently swallowed by .gitignore.
[... 4 findings printed ...]
EXIT=1
```

Added `.specify/extensions/superspec/` to `EXCLUDED_PATH_PREFIXES` in `scripts/pre_build/check_gitignore_swallow.sh`, with an in-source comment citing this exact investigation (never a silent exclusion). Also recorded the classification directly in the `.gitignore` comment block for that rule (per BOB-244's own stated acceptance criteria), and wrote the previously-missing companion doc `docs/scripts/check_gitignore_swallow.md` (§11.4.18 — the script's own header names this doc; it never existed).

GREEN confirmed:

```
$ bash scripts/pre_build/check_gitignore_swallow.sh .
BOB-212 swallow-guard: OK – no first-party source files are
```

```
silently swallowed by .gitignore.  
EXIT=0
```

Regression verification (no weakening of real detection)

Both existing test suites for this gate re-run clean, unmodified, proving the new exclusion does not weaken detection of a genuine swallow elsewhere:

```
$ bash tests/pre_build/test_check_gitignore_swallow.sh  
-- case 1: golden-GOOD (swallowed first-party source file) --  
PASS: case 1: guard exited non-zero (1) and named the file +  
gitignore:<N> rule  
-- case 2: golden-BAD / false-positive guard (only secret-shaped  
files) --  
PASS: case 2: guard exited zero on a tree whose ONLY  
ignored+untracked files are secret-shaped  
-- case 3: genuinely clean tree (nothing ignored+untracked) --  
PASS: case 3: guard exited zero on a clean tree with no  
ignored+untracked files  
-- case 4: swallowed file under an excluded root (not first-party)  
--  
PASS: case 4: guard exited zero -- excluded-root swallows are  
correctly NOT first-party  
-- case 5: fail-closed on unresolvable input ($11.4.252) --  
PASS: case 5a: guard exited 2 on a nonexistent repo root  
PASS: case 5b: guard exited 2 on a directory that is not a git  
repo  
-- case 6: paired $1.1 mutation (guard broken -> this test detects  
it) --  
PASS: case 6a: mutated (always-pass) guard exits 0 on the  
swallowed-file tree -- mutation confirmed live  
PASS: case 6b: mutated guard correctly FAILS the golden-good  
assertion (this test would catch a broken guard)  
PASS: case 6c: restored (real, unmutated) guard passes the golden-  
good assertion again  
RESULT: PASS -- all cases behaved as specified (0 failures).  
  
$ bash tests/security/test_gitignore_swallow_is_loud.sh  
-- A2: does a swallow-guard exist and refuse loudly, naming rule +  
file? --  
[ok] guard refused (exit 1) and named both the file and  
the .gitignore rule  
-- A3: does ANY signal distinguish 'swallowed' from 'never  
authored'? --  
[ok] a signal reaches the author  
-- GOLDEN-FALSE ($11.4.201(1)/$11.4.10): real secrets MUST stay  
ignored --
```

```
[ok]      all 15 secret-bearing shapes remain ignored
RESULT: PASS (exit 0) -- the swallow is LOUD (or the file commits
normally).
```

```
bash -n scripts/pre_build/check_gitignore_swallow.sh → clean parse.
```

Test type declared

Bash unit/regression test (existing suite, unmodified, re-run against the fix) + a real repo-state investigation (`diff -q` against the tracked submodule) as the oracle proving the exclusion is evidence-backed rather than assumed.

Closure

Fixed. Acceptance criterion satisfied: “CM-GITIGNORE-SWALLOW-GUARD passes clean (0 findings) OR the 4 files are explicitly, evidence-backed classified as vendored-and-correctly-ignored with that classification recorded in the `.gitignore` comment itself” — the gate now passes clean AND the classification is recorded in `.gitignore`.